

# Capítulo 2

## Capa de Aplicación: La Web Parte 2

|             |
|-------------|
| Application |
| Transport   |
| Network     |
| Link        |
| Physical    |

# La Web: Metas

- **Vamos a estudiar:**

- Páginas Web, sitios y aplicaciones web
- Infraestructura para la web
- Identificación de páginas web y formularios
- Navegadores (browsers) y servidores Web
- **Protocolo de comunicación para la web**
- Manejo de estado de sesión en la web
- Páginas dinámicas

# Bloque 7: Protocolo para la web

- **Introducción:** Conociendo ambos extremos —navegador y servidor— estamos listos para derivar el protocolo que los conecta.
  - Analizaremos qué problemas debe resolver, qué tipos de mensajes intercambia y cómo evolucionó para mejorar rendimiento y paralelismo.
  - Haremos ejemplos con el protocolo HTTP ampliamente adoptado.

# Bloque 7: Protocolo para la web

- **Metas:** derivar un protocolo para la web
- **¿Qué cosas debería considerar un protocolo para la web?**
  - **Ayuda:** recordar lo que es un protocolo y tener en cuenta que estamos en la web.

# Bloque 7: Protocolo para la web

- **Metas:** derivar un protocolo para la web
- **¿Qué cosas debería considerar un protocolo para la web?**
  - Formatos de mensajes para pedido de páginas estáticas, objetos o de ejecución de páginas dinámicas.
  - Formatos de mensajes para operaciones que mantienen sistema de archivos en servidor web.
  - Formato de mensajes para enviar páginas a browser.
  - Manejo de información de estado de sesión.
  - Seguridad: encriptación de mensajes.
  - Retroalimentación cuando no se pueden responder los pedidos
  - Comunicación confiable.
- **¿En qué protocolo de capa de transporte conviene apoyarse para definir un protocolo para la web?**
  - Justificar la respuesta.

# Bloque 7: Protocolo para la web

- **Metas:** derivar un protocolo para la web
- **¿Qué cosas debería considerar un protocolo para la web?**
  - Formatos de mensajes para pedido de páginas estáticas, objetos o de ejecución de páginas dinámicas.
  - Formatos de mensajes para operaciones que mantienen sistema de archivos en servidor web.
  - Formato de mensajes para enviar páginas a browser.
  - Manejo de información de estado de sesión.
  - Seguridad: encriptación de mensajes.
  - Feedback cuando no se pueden responder los pedidos
  - Comunicación confiable.
- **¿En qué protocolo de capa de transporte conviene apoyarse para definir un protocolo para la web?**
  - Conviene TCP porque una respuesta puede requerir varios paquetes y al fragmentar la información aparecen los problemas que resuelve TCP.

# Bloque 7: Protocolo para la web

- ¿Qué tipos de mensajes necesitamos para un protocolo para la web?
  - **Ayuda:** considerar distintos casos de comunicación

# Bloque 7: Protocolo para la web

- ¿Qué tipos de mensajes necesitamos para un protocolo para la web?
  - Mensajes de pedido y mensajes de respuesta
- Imagina que tienes que diseñar desde cero cómo un navegador usa conexiones TCP para descargar una página y todos sus recursos. ¿Qué estrategias posibles se te ocurren para abrir, reutilizar o cerrar conexiones, y qué ventajas o problemas tendría cada una?



# Bloque 7: Protocolo para la web

- ¿Qué tipos de mensajes necesitamos para un protocolo para la web?
  - Mensajes de pedido y mensajes de respuesta
- Imagina que tienes que diseñar desde cero cómo un navegador usa conexiones TCP para descargar una página y todos sus recursos. ¿Qué estrategias posibles se te ocurren para abrir, reutilizar o cerrar conexiones, y qué ventajas o problemas tendría cada una?
  - **No persistencia**: Una conexión por recurso, y una conexión para la página (en HTTP 1.0).
  - **Persistencia**: Una sola conexión para todo (en HTTP 1.1).
    - Los pedidos son procesados en orden y los resultados se mandan en orden.

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Imagina que un recurso tarda mucho (p.ej. una media muy pesada). **¿Qué sucede?**

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Imagina que un recurso tarda mucho (p.ej. una media muy pesada). **¿Qué sucede?**
  - Si una respuesta se demora, bloquea todas las demás.
  - Por lo tanto hay que evitar que esto suceda.
- **Proponer ideas de solución a este problema.**

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Imagina que un recurso tarda mucho (p.ej. una media muy pesada). **¿Qué sucede?**
  - Si una respuesta se demora, bloquea todas las demás.
  - Por lo tanto hay que evitar que esto suceda.
- **Proponer ideas de solución a este problema.**
- **Alterar el orden de las respuestas a los pedidos.**
  - Para que las respuestas rápidas no tengan que esperar; así puede mostrarse lo más posible de manera lo más rápida posible.
- **Mezclar pedacitos de varias respuestas**
  - así se evita que una respuesta grande bloquee a las demás en la misma conexión; el navegador puede recibir primero lo que necesita antes.
- **Indicar prioridades para pedidos de recursos**
  - Evita que recursos pesados eviten la preparación de recursos livianos.

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Supongamos que modificamos el protocolo para reducir la cantidad de pedidos de recursos (o sea, el servidor puede enviar recursos sin que el cliente los pida uno a uno)  
**¿Cuál ventaja tiene hacer esto?**

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Supongamos que modificamos el protocolo para reducir la cantidad de pedidos de recursos (o sea, el servidor puede enviar recursos sin que el cliente los pida uno a uno)  
**¿Cuál ventaja tiene hacer esto?**
- Cada pedido tradicional implica:
  1. El cliente detecta que necesita un recurso.
  2. Envía un request.
  3. Espera un RTT.
  4. Recién entonces empieza a recibir la respuesta.
- Si eliminas el paso 1–2 (porque el servidor ya sabe qué recursos vas a necesitar), te ahorras al menos un RTT por recurso.
  - En redes móviles o de alta latencia, esto es oro.

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Supongamos que modificamos el protocolo para indicar prioridades para pedidos de recursos **¿Qué beneficios tiene hacer esto?**

# Bloque 7: Protocolo para la web

- Asumir que usamos una sola conexión persistente y enviamos los pedidos en orden, recibiendo también las respuestas en orden.
- Supongamos que modificamos el protocolo para indicar prioridades para pedidos de recursos **¿Qué beneficios tiene hacer esto?**
- El servidor puede procesar antes los pedidos prioritarios, generar antes sus datos y tenerlos listos para enviar apenas le toque el turno.
  - Esto reduce el tiempo hasta que el cliente recibe lo crítico;
  - permite reducir latencia percibida sin romper el orden de las respuestas.
- Evitar que recursos pesados retrasen la preparación de recursos livianos.



# Bloque 7: Protocolo para la web

- **Solución implementada por HTTP 2.0:**

- Enviar muchos pedidos y muchas respuestas en paralelo dentro de una misma conexión, intercalando fragmentos (“frames”) de cada uno.
- El servidor web puede enviar recursos antes de que el cliente los pida.
- El cliente puede indicar prioridades para el envío de recursos.

# Bloque 7: Protocolo para la web

## Tipos de pedidos

- ¿Qué tipos de pedidos hay a un servidor en un protocolo para la web?

# Bloque 7: Protocolo para la web

## Tipos de pedidos

- **¿Qué tipos de pedidos hay a un servidor en un protocolo para la web?**
  - Pedido de página estática
  - Pedido de página dinámica
  - Pedido de datos
  - Pedido de operación sobre sistema de archivos del servidor web (agregar página, borrar página)

# Bloque 7: Protocolo para la web

## Formato de pedidos

- **Ejercicio:** Asumir que se usa un solo formato de mensaje de pedido para todos los pedidos. **¿Qué tipos de informaciones conviene poner en un mensaje de pedido en un protocolo para la web?** (ayuda: considerar los diferentes tipos de pedidos)

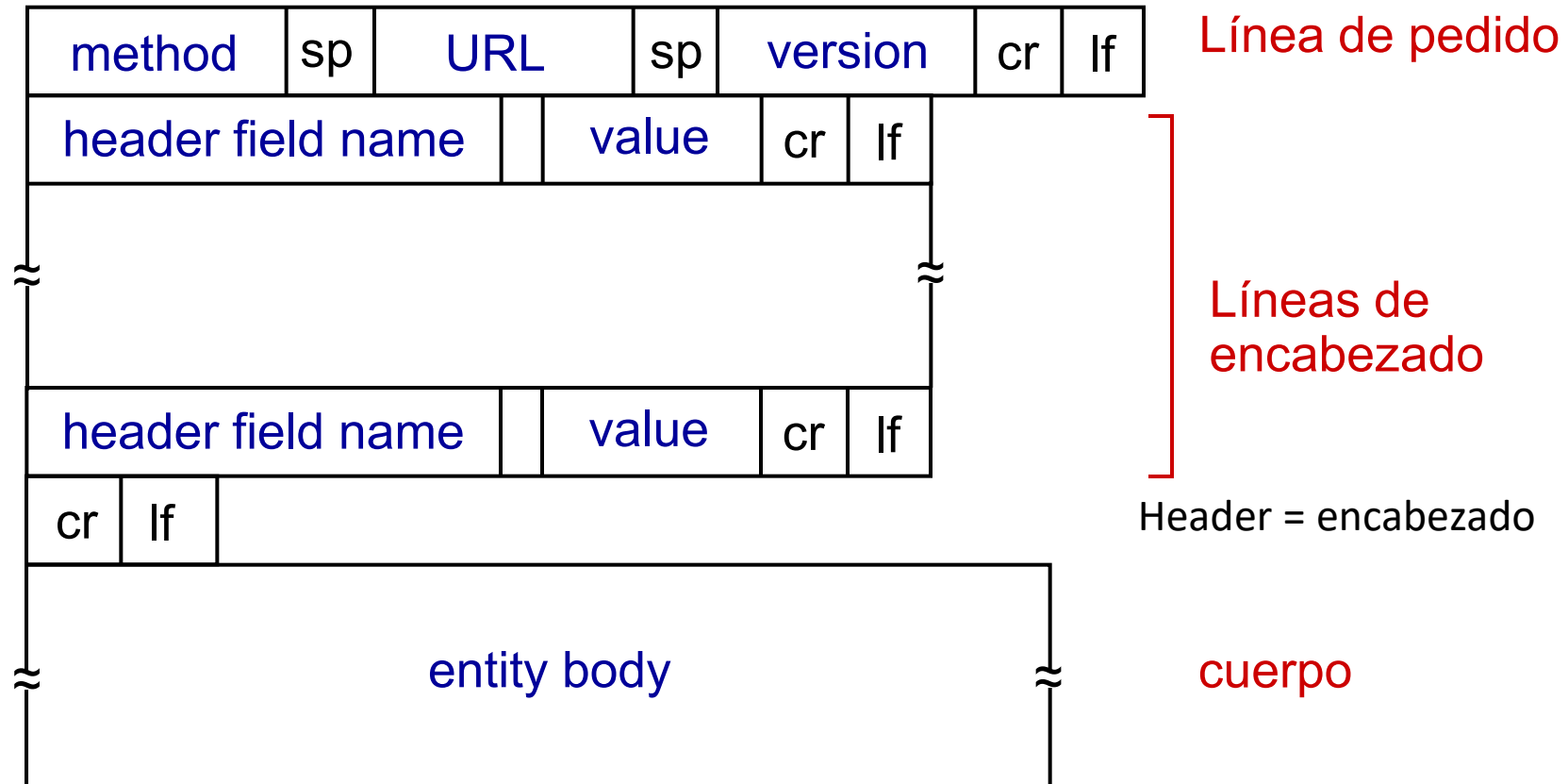
# Bloque 7: Protocolo para la web

## Formato de pedidos

- **Ejercicio:** Asumir que se usa un solo formato de mensaje de pedido para todos los pedidos. **¿Qué tipos de informaciones conviene poner en un mensaje de pedido en un protocolo para la web?** (ayuda: considerar los diferentes tipos de pedidos, diferentes versiones del protocolo)
  - Campo para version del protocolo
  - Campo URL (para pedido de todo tipo de páginas.)
  - Campo que indica para qué es el pedido (operación sobre Sistema de archivos, pedido de página, etc.)
  - Campo de datos (para enviar parámetros de formulario, para enviar documento a subir en servidor web)
    - De tamaño variable.
  - Campos de informaciones adicionales (con nombre de la información y valor de la misma).
    - La cantidad de estos campos varia de un pedido a otro.

# Bloque 7: Protocolo para la web

## Formato de pedidos



- ❑ A la línea de solicitud le pueden seguir líneas adicionales llamadas **encabezados de solicitud**.

# Bloque 7: Protocolo para la web

## Formato de pedidos

request line  
(GET, POST,  
HEAD commands)

header  
lines

carriage return,  
line feed at start  
of line indicates  
end of header lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return character

line-feed character

The diagram illustrates the structure of an HTTP request. It shows a request line followed by several header lines, and then a blank line. Blue arrows point from descriptive text to specific parts of the request: one points to the request line, another to the header lines, and a third to the blank line. Additionally, two arrows point to the backslash and 'r' characters in the first header line, identifying them as carriage return and line-feed characters respectively.

# Bloque 7: Protocolo para la web

## Tipos de respuestas

- **¿Que tipos de respuestas hay a un pedido?** (**ayuda:** considerar tipos de respuestas para tipos de pedidos)



# Bloque 7: Protocolo para la web

## Tipos de respuestas

- ¿Que tipos de respuestas hay a un pedido?
  - Página web (estática o generada)
  - Datos
  - Resultado de operación en sistema de archivos del servidor web.

# Bloque 7: Protocolo para la web

## Formato de las respuestas

- **Ejercicio:** ¿Qué tipos de informaciones conviene poner en un mensaje de respuesta del servidor web en un protocolo para la web? (**ayuda:** considerar tipos de respuestas)

# Bloque 7: Protocolo para la web

## Formato de las respuestas

- **Ejercicio:** ¿Qué tipos de informaciones conviene poner en un mensaje de respuesta del servidor web en un protocolo para la web?
  - Mensaje de feedback sobre el pedido
  - Campo con documento enviado (página estática o página generada, datos)
    - De tamaño variable.
  - Campos con informaciones adicionales (con nombre de la información y valor de la misma)
    - Cantidad variable de estos.
    - **Por ejemplo**, datos del servidor, tipo de contenido de la respuesta, longitud del contenido.

# Bloque 7: Protocolo para la web

## Formato de las respuestas

- **Partes de una respuesta HTTP**
  1. **Línea de estado.**
  2. (opcional) **encabezados de respuesta.**
  3. Luego vienen el cuerpo de la respuesta:
    - p.ej. página estática usando archivo en formato HTML.
    - P.ej. datos pedidos usando documento en formato XML.

# Bloque 7: Protocolo para la web

## Formato de las respuestas

Ejemplo de respuesta HTTP

The start of the output of  
*[www.ietf.org/rfc.html](http://www.ietf.org/rfc.html)*.

```
Trying 4.17.168.6...
Connected to www.ietf.org.
Escape character is '^]'.
HTTP/1.1 200 OK
Date: Wed, 08 May 2002 22:54:22 GMT
Server: Apache/1.3.20 (Unix) mod_ssl/2.8.4 OpenSSL/0.9.5a
Last-Modified: Mon, 11 Sep 2000 13:56:29 GMT
ETag: "2a79d-c8b-39bce48d"
Accept-Ranges: bytes
Content-Length: 3211
Content-Type: text/html
X-Pad: avoid browser bug
```

```
<html>
<head>
<title>IETF RFC Page</title>
```

```
<script language="javascript">
function url() {
  var x = document.form1.number.value
  if (x.length == 1) {x = "000" + x }
  if (x.length == 2) {x = "00" + x }
  if (x.length == 3) {x = "0" + x }
  document.form1.action = "/rfc/rfc" + x + ".txt"
  document.form1.submit
}
</script>
```

```
</head>
```

# Bloque 7: Protocolo para la web

- **Recapitulación:** Vimos qué debe contemplar un protocolo web, por qué se apoya en TCP, cómo funcionan las conexiones persistentes, qué problemas resuelve HTTP/2, y qué información contienen los pedidos y respuestas.
- Con el protocolo claro, podemos estudiar cómo se mantiene el estado de sesión en un entorno sin estado como HTTP.

# La Web: Metas

- **Vamos a estudiar:**

- Páginas Web, sitios y aplicaciones web
- Infraestructura para la web
- Identificación de páginas web y formularios
- Navegadores (browsers) y servidores Web
- Protocolo de comunicación para la web
- Manejo de estado de sesión en la web
- Páginas dinámicas

# Bloque 8: Manejo de estado de sesión

- **Propósito:**
  - derivar cómo manejar el estado de sesión en la web y
  - entender la solución de las cookies.
- **¿Al usar una aplicación web HTTP recuerda algo entre una petición y otra?** (ayuda: por ejemplo en app de comercio electrónico)



# Bloque 8: Manejo de estado de sesión

- **Introducción:** HTTP no recuerda nada entre un pedido y otro, pero las aplicaciones sí necesitan memoria.
  - En este bloque veremos cómo se construye el estado de sesión, dónde se almacena y qué desafíos plantea su mantenimiento.

# Bloque 8: Manejo de estado de sesión

- **Propósito:**
  - derivar cómo manejar el estado de sesión en la web y
  - entender la solución de las cookies.
- **¿Al usar una aplicación web HTTP recuerda algo entre una petición y otra?**
  - A veces sí. Por ejemplo, el contenido del carrito de compras.
- Si no recuerda nada, **¿cómo hace la aplicación para mantener continuidad?**

# Bloque 8: Manejo de estado de sesión

- **Propósito:**
  - derivar cómo manejar el estado de sesión en la web y
  - entender la solución de las cookies.
- **¿Al usar una aplicación web HTTP recuerda algo entre una petición y otra?**
  - A veces sí. Por ejemplo, el contenido del carrito de compras.
- Si no recuerda nada, **¿cómo hace la aplicación para mantener continuidad?**
  - Debe guardar algo en algún lugar.

# Bloque 8: Manejo de estado de sesión

- Ejemplos de cosas que puede recordar una aplicación web:

| Necesidad                                 | Dato que se guarda     |
|-------------------------------------------|------------------------|
| Saber quien soy                           | userId: "12345"        |
| Mantenerme autenticado                    | isAuthenticated: true  |
| Recordar en qué parte del proceso estoy   | currentStep: 2         |
| Conservar mis preferencias                | Language: "es-AR"      |
| Recordar contenido del carrito de compras | Cart: "id12-2, id35-4" |

# Bloque 8: Manejo de estado de sesión

- **¿Qué es el estado de sesión de una aplicación web?**
  - Es la información que la aplicación guarda sobre el usuario entre una petición y otra para mantener continuidad, personalización y seguridad en su experiencia.
- **¿Cómo nombrar y dar valor a una información de estado de sesión?**

# Bloque 8: Manejo de estado de sesión

- **¿Qué es el estado de sesión de una aplicación web?**
  - Es la información que la aplicación guarda sobre el usuario entre una petición y otra para mantener continuidad, personalización y seguridad en su experiencia.
- **¿Cómo nombrar y dar valor a una información de estado de sesión?**
  - Usando un par: nombre de la información y valor.
- **¿Dónde hace falta saber el estado de sesión? ¿Del lado del cliente o del lado del servidor?**
  - **Ayuda:** considerar el ejemplo de un carrito de compras en una app de comercio electrónico.

# Bloque 8: Manejo de estado de session

## Almacenamiento del estado de sesión

- **¿Dónde almacenar el estado de sesión?**
  - Considerar alternativas y describir cada una.

# Bloque 8: Manejo de estado de session

## Almacenamiento del estado de sesión

- ¿Dónde almacenar el estado de sesión?
  - **Del lado del browser:**
    - Por ejemplo: las informaciones de estado se llaman cookies y se guardan en un directorio de cookies.
  - **Del lado del servidor:**
    - En base de datos tradicional (en disco)
      - **Por ejemplo:** base de datos relacional basada en SQL.
    - En memoria RAM, no en disco (esto es mucho más veloz).
      - **Por ejemplo: Redis** (puede estar en la máquina del servidor web como otro proceso, o en otra máquina o en la nube)
        - el servidor genera un session ID;
        - ese ID se guarda en una cookie;
        - el servidor guarda en Redis un objeto asociado a ese ID con las informaciones de estado de sesión.



# Bloque 8: Manejo de estado de sesión

- Asumir que la información de estado se guarda en el cliente, y se quiere hacer un pedido al servidor.
- **¿Cómo identificar la información de estado de sesión que hay que mandar junto con un pedido al servidor web? Ayuda:**  
Considerar que:
  - No todas las informaciones de sesión son para la misma aplicación web que corre en el servidor.
  - No todas las informaciones de sesión para la aplicación son para todas las páginas dinámicas.

# Bloque 8: Manejo de estado de sesión

- Asumir que la información de estado se guarda en el cliente, y se quiere hacer un pedido al servidor.
- **¿Cómo identificar la información de estado de sesión que hay que mandar junto con un pedido al servidor web?**
  - Para una información de estado de sesión **guardar el domino** para el que se va a usar.
  - Para una información de estado de sesión **guardar el camino y nombre de página dinámica** que se va a usar.
    - Con los cookies solo se **guarda el camino**.

# Bloque 8: Manejo de estado de sesión

- Asumir que la información de estado de sesión se guarda en el cliente.
- Asumir que el servidor web modificó cierta información de estado de sesión como consecuencia de un pedido del cliente.
- ¿Qué hay que hacer para tener esa información de estado de sesión actualizada en el cliente?

# Bloque 8: Manejo de estado de sesión

- Asumir que la información de estado de sesión se guarda en el cliente.
- Asumir que el servidor web modificó cierta información de estado de sesión como consecuencia de un pedido del cliente.
- **¿Qué hay que hacer para tener esa información de estado de sesión actualizada en el cliente?**
  - El servidor debe enviar explícitamente esa información en la respuesta;
  - el cliente la reemplaza en su almacenamiento local;
  - el cliente debe usar ese nuevo estado en las peticiones.

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- ¿Qué pasa si la información de estado de sesión vive en el cliente y no nos preocupamos por su duración? (**ayuda:** indicar problemas de seguridad, problemas desincronización entre el cliente y el servidor, considerar consumo de recursos, considerar experiencia del usuario)

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- ¿Qué pasa si la información de estado de sesión vive en el cliente y no nos preocupamos por su duración?
  - Usuarios que no deberían estar autenticados siguen teniendo acceso.
  - Alguien puede llegar a robar información de estado de sesión y usarla porque no expira.
  - El servidor puede rechazar pedidos porque la sesión ya expiró pero el cliente cree que sigue vigente.
  - Se comienza a llenar el cliente de información de sesiones viejas.
  - El usuario puede quedar medio logueado: la interfaz cree que está autenticado, pero el servidor no.
  - Formularios pueden perderse sin explicación.

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- **Conclusión:** Si no controlamos la duración del estado de sesión, aparecen problemas de seguridad, datos viejos, sesiones que nunca mueren, inconsistencias entre cliente y servidor y una mala experiencia de usuario.
- **¿Cómo expresar la duración de una información de estado de sesión?**

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- **Conclusión:** Si no controlamos la duración del estado de sesión, aparecen problemas de seguridad, datos viejos, sesiones que nunca mueren, inconsistencias entre cliente y servidor y una mala experiencia de usuario.
- **¿Cómo expresar la duración de una información de estado de sesión?**
  - Guardar **el día y la hora en que va expirar** una información de estado de sesión.



# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- Supongamos que la información de estado de sesión se guarda en un folder del cliente.
  - ¿Qué hacer cuando expira una información de estado de sesión?

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- Supongamos que la información de estado de sesión se guarda en un folder del cliente.
  - ¿Qué hacer cuando expira una información de estado de sesión?
  - El browser puede borrar esa información (borrado periódico).
  - ¿Qué hacer con información de estado de sesión que no tiene fecha de expiración?

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- Supongamos que la información de estado de sesión se guarda en un folder del cliente.
  - **¿Qué hacer cuando expira una información de estado de sesión?**
  - El browser puede borrar esa información (borrado periódico).
  - **¿Qué hacer con información de estado de sesión que no tiene fecha de expiración?**
  - El browser puede eliminar esa información de estado de sesión cuando el usuario sale del browser.
  - **¿Qué hacer si el servidor quiere que una información de estado de sesión que aun no expiró sea eliminada?**

# Bloque 8: Manejo de estado de session

## Duración de estado de sesión

- Supongamos que la información de estado de sesión se guarda en un folder del cliente.
  - **¿Qué hacer cuando expira una información de estado de sesión?**
  - El browser puede borrar esa información (borrado periódico).
  - **¿Qué hacer con información de estado de sesión que no tiene fecha de expiración?**
  - El browser puede eliminar esa información de estado de sesión cuando el usuario sale del browser.
  - **¿Qué hacer si el servidor quiere que una información de estado de sesión que aun no expiró sea eliminada?**
  - El servidor al responder envía la información de estado de sesión con una fecha caducada.

# Bloque 8: Manejo de estado de sesión

- **Ejercicio integrador:** Supongamos que se maneja información de carrito de compras. Describir todos los pasos que se hacen con esa información de estado de sesión. Para esto considerar:
  - Del lado del cliente se pueden agregar ítems al carrito de compras.
    - Para esto el cliente navega categorías y productos.
  - El usuario puede hacer la compra de los ítems del carrito.
  - Incluir mensajes de pedidos y de respuesta y cuándo se incluye la información del carrito junto con ellas.
  - Considerar además creación, actualización, vaciado del carrito.

# Bloque 8: Manejo de estado de sesión

- **Recapitulación:** Comprendimos qué es el estado de sesión, qué datos puede incluir, dónde almacenarlo (cliente, servidor o ambos), cómo se envía y actualiza, y por qué es crucial controlar su duración para evitar problemas de seguridad y consistencia.
- Con esto ya podemos profundizar en cómo las páginas dinámicas usan estos mecanismos para generar contenido personalizado.

# La Web: Metas

- **Vamos a estudiar:**
  - Páginas Web, sitios y aplicaciones web
  - Infraestructura para la web
  - Identificación de páginas web y formularios
  - Navegadores (browsers) y servidores Web
  - Protocolo de comunicación para la web
  - Manejo de estado de sesión en la web
  - Páginas dinámicas

# Bloque 9: Páginas dinámicas

- **Introducción:** Finalmente, integraremos todo lo aprendido para entender cómo se generan páginas dinámicas:
  - cómo reciben parámetros, cómo ejecutan lógica y cómo producen HTML personalizado.
  - Además, estudiaremos lo esencial del lenguaje PHP.



# Bloque 9: Páginas dinámicas

## Su comportamiento

- **Repaso:** Páginas dinámicas son páginas web generadas por programas que se ejecutan del lado del servidor.
  - Pueden ser programas en lenguaje de programación o de scripting.
- **¿Cuáles son los pasos gruesos para generar una página dinámica?**  
(ayuda: considerar desde que el pedido de página dinámica llega al servidor web, que se accede a base de datos. No entrar en detalles de procesamiento de pedido HTTP)

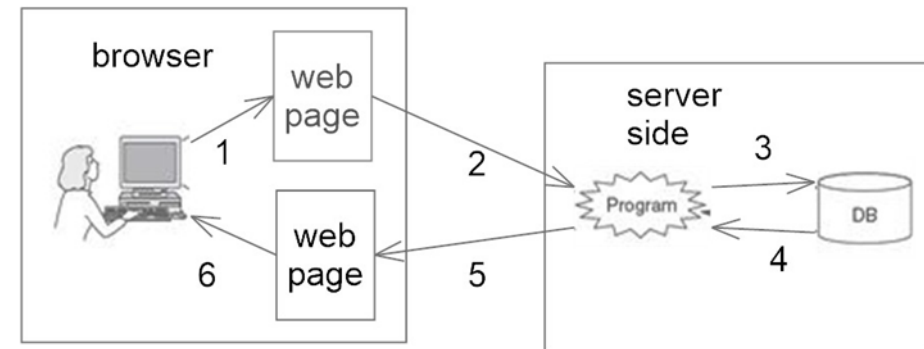
# Bloque 9: Páginas dinámicas

## Su comportamiento

- **Pasos gruesos para generar una página dinámica:**

1. El mensaje es entregado a un programa para procesarlo.
2. El programa solicita la información a un servidor de bases de datos.
3. El servidor de bases de datos responde con la información requerida.
4. El programa genera una página HTML personalizada y la envía al cliente.

5. El browser muestra la página recibida al usuario.



# Bloque 9: Páginas dinámicas

## Su comportamiento

- **¿Qué tareas finas suelen hacer las páginas dinámicas?** (ayuda: entrar en detalles de procesamiento de pedido – p.ej. en HTTP)

# Bloque 9: Páginas dinámicas

## Su comportamiento

- **¿Qué tareas finas suelen hacer las páginas dinámicas?**
  - Procesar parámetros de formularios
  - Procesar información especial del pedido (p.ej. encabezados de pedido HTTP)
  - Pedir datos a fuentes de datos (e.g. bases de datos)
  - Generar página web con los datos recibidos (p.ej. en HTML).
  - Generar feedback e informaciones especiales de respuesta (p.ej. encabezados de respuesta HTTP).
- **Tecnologías para producir páginas dinámicas**
  - PHP, Java Server Pages , etc.

# Bloque 9: Páginas dinámicas

## PHP

- **PHP – Preprocesador de hipertexto.**
  - Tienen la apariencia de páginas HTML donde se insertan comandos especiales.
  - Esos comandos son interpretados por un servidor web.
- Si Ud fuera a construir un lenguaje como PHP
- **¿Para hacer qué cosas necesitaría definir sintaxis?**
- **Ayuda:** recordar http, manejo de estado de sesión, etc.

# Bloque 9: Páginas dinámicas

## PHP

- **PHP – Preprocesador de hipertexto.**
  - Tienen la apariencia de páginas HTML donde se insertan comandos especiales.
  - Esos comandos son interpretados por un servidor web.
- Si Ud fuera a construir un lenguaje como PHP
- **¿Para hacer qué cosas necesitaría definir sintaxis?**
  - Para acceder a campos de formularios
  - Acceso a campos de información especial de pedido (encabezados HTTP)
  - Definición de información especial de respuesta (encabezados de respuesta HTTP)
  - Acceso a información de estado de sesión (i.e. de cookies)
  - Actualización o definición de información de estado de sesión (i.e. de cookies)
  - Para mostrar datos en pantalla.
- Leer los detalles en archivo anexo.

# Bloque 9: Páginas dinámicas PHP

Página web que obtiene input de formulario y llama un programa de servidor

```
<html><body>
  <form action="welcome.php" method="post">
    Name: <input type="text" name="name"><br>
    E-mail: <input type="text" name="email"><br>
    <input type="submit">
  </form>
</body> </html>
```

Programa PHP del servidor que crea una página web personalizada

```
<html><body>
  Welcome <php echo $_POST["name"]; ?><br>
  Your email address is: <?php echo $_POST["email"]; ?>
</body></html>
```

Página web resultante (para inputs "John" y "john.doe@example.com")

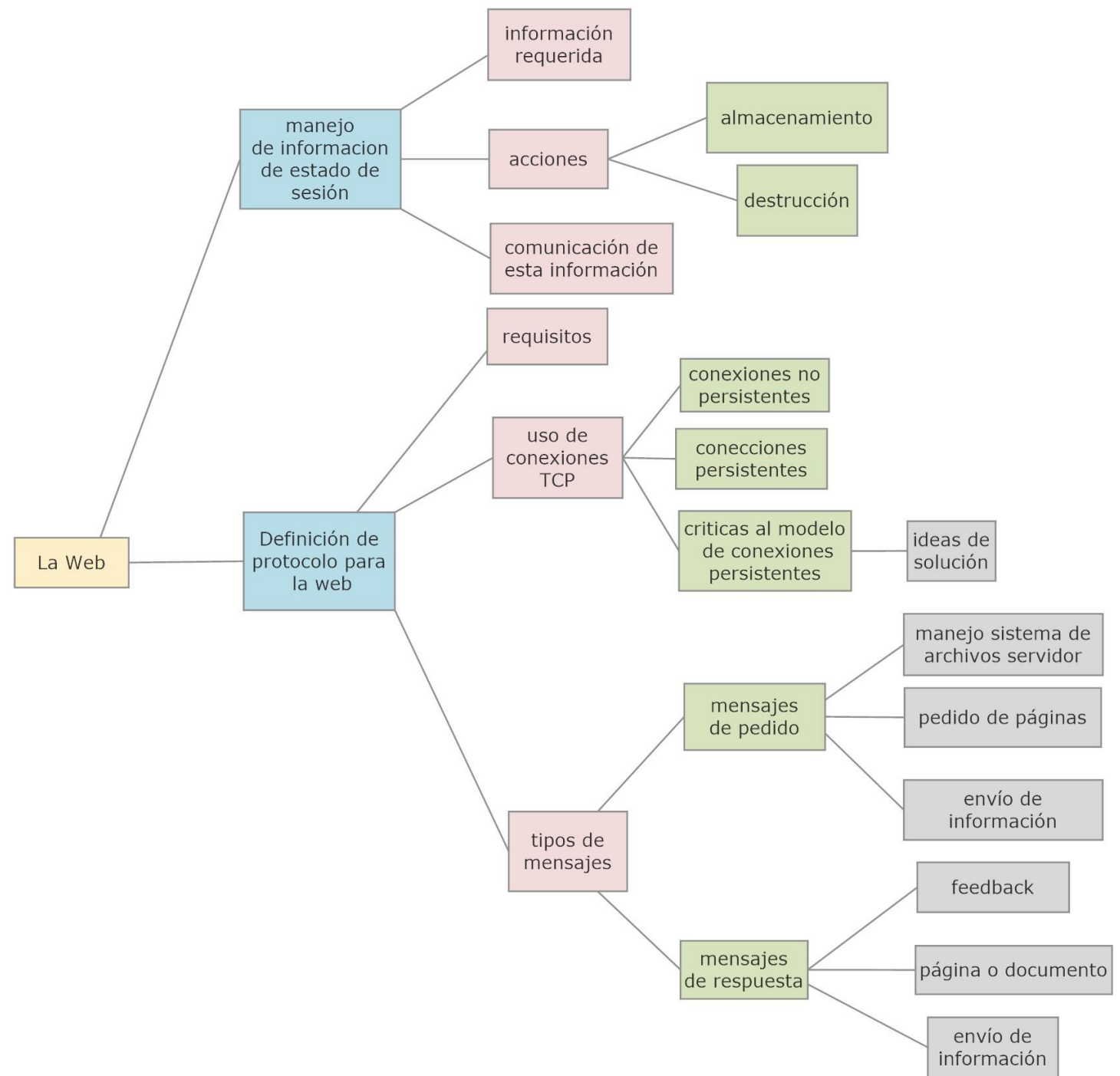
```
<html><body>
  Welcome John
  Your email address is john.doe@example.com
</body></html>
```

# Bloque 9: Páginas dinámicas

- **Recapitulación:** Vimos cómo se generan páginas dinámicas, qué tareas realizan (procesar parámetros, consultar bases de datos, generar HTML, enviar encabezados) y qué tecnologías se usan como PHP.
- Con esto se completa el recorrido desde la estructura básica de la Web hasta la generación dinámica de contenido.



# La Web



# La Web

- A lo largo de este recorrido vimos cómo se integran páginas, navegadores, servidores, protocolos y mecanismos de estado para formar el ecosistema web.
  - Comprender estas piezas no solo permite analizar la Web actual, sino también diseñar y construir aplicaciones que aprovechen su arquitectura de manera eficiente.